DSA Assignment 2

Sliding Window Technique

Name: Ankitraj Jha

Roll No: 150096725050

Class: demis hassabis

Subject:DSA 2

GitHub Link:
https://github.com/Ankitraj-17/DSA-Assignments/tree/main/Assignment%202

1. Aim

To solve two problems using the sliding window technique: (A) the maximum sum of K consecutive elements, which uses a fixed-size window, and (B) the longest subarray whose sum is at most K, which uses a variable-size window. For each problem I also wrote a simple method without sliding window, so the two can be compared. All programs are written in C++, Java and Python.

2. Problem Statements

Part A: Maximum sum of a subarray of size K (fixed window)

An integer array of length n and a window size K are given, with 1 <= K <= n. Find the maximum sum of any K consecutive elements. The array can have negative numbers.

Example: a = [2, 1, 5, 1, 3, 2], K = 3. The answer is 9, from the subarray [5, 1, 3].

Part B: Longest subarray with sum at most K (variable window)

An array of non-negative integers and a non-negative limit are given. Find the longest continuous subarray whose sum is less than or equal to the limit, and give its length with the start and end index. If many subarrays have the same longest length, any one is accepted. If no non-empty subarray works, the length is 0.

Example: a = [1, 2, 1, 1, 3], limit = 4. The answer is length 3, from [1, 2, 1] (indices 0 to 2) or [2, 1, 1] (indices 1 to 3).

Input rules

- Part A needs 1 <= K <= n. If K is outside this range, the functions print an error message and return the smallest integer instead of a sum.
- Part B works only when every array value and the limit are non-negative. This is the condition of the problem, and the sliding window method must not be used for arrays with negative values (the

reason is in section 4.2). The program does not check for negative values, so the input has to be correct.

- In this report the limit of Part B is called limit, so it is not mixed up with the window size K of Part A.

3. Algorithms

3.1 Part A, simple method

1. Check that K is between 1 and n. If not, print an error and stop.
2. Set maxSum to the smallest possible integer.
3. For each starting index i from 0 to n-K, add the K numbers a[i] to a[i+K-1] from the beginning.
4. If this sum is greater than maxSum, store it in maxSum.
5. After all starting indices, return maxSum.

3.2 Part A, sliding window method

1. Check that K is between 1 and n. If not, print an error and stop.
2. Add the first K numbers once. This is windowSum, and it is also the starting value of maxSum.
3. For i from K to n-1: windowSum = windowSum + a[i] - a[i-K]. Here a[i] enters the window and a[i-K] leaves it.
4. If windowSum is greater than maxSum, store it in maxSum.
5. After the loop, return maxSum.

3.3 Part B, simple method (brute force)

1. Set best length = 0, best start = -1, best end = -1.
2. For each starting index i, set sum = 0 and add a[i], a[i+1], ... one by one.
3. If the sum becomes more than the limit, stop the inner loop. The numbers are not negative, so the sum can only grow.
4. Otherwise, if the length j-i+1 is more than the best length, store the length with i and j.
5. After all starting indices, return the best length, start and end.

3.4 Part B, sliding window method

1. Set left = 0, sum = 0, best length = 0, best start = -1, best end = -1.
2. For right from 0 to n-1, add a[right] to the sum. This is the expand step.
3. While the sum is more than the limit (and left <= right), subtract a[left] and increase left by 1. This is the contract step.
4. If the window is not empty and its length right-left+1 is more than the best length, update the best length, start and end.
5. After the loop, return the best length, start and end.

4. How the Windows Move

4.1 Fixed window

The first window covers indices 0 to K-1. Its sum is calculated once, with a loop. After that, every move to the right changes only two elements: a[i] enters on the right and a[i-K] leaves on the left. The other K-1 elements stay inside, so they are not added again. This is why the sum changes with just one addition and one subtraction at every slide.

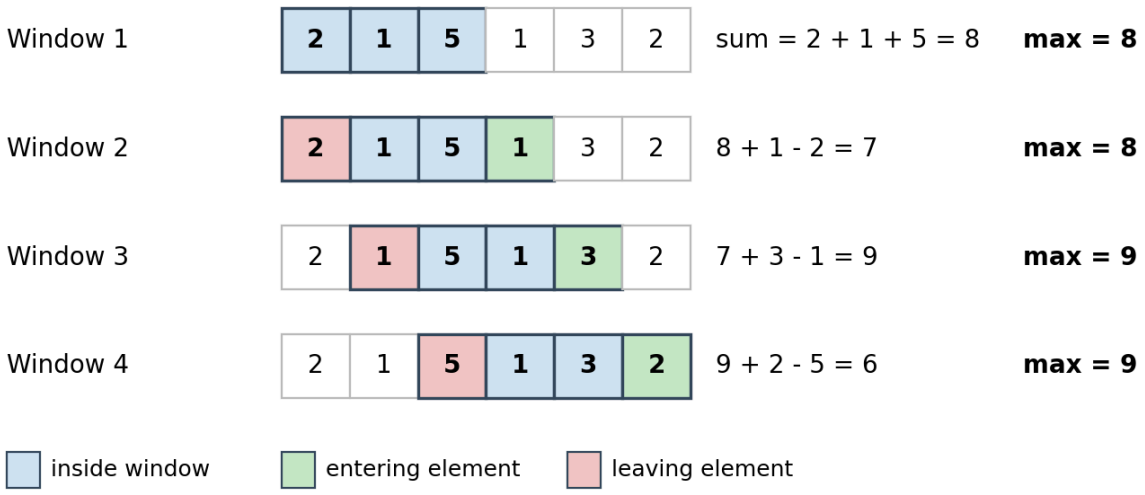


Figure 2: Fixed window for a = [2,1,5,1,3,2] and K = 3 (maximum sum = 9)

maxSum starts from the first window's sum and not from 0. If it started from 0, an array with only negative numbers, such as [-4, -2, -7] with K = 2, would wrongly give 0 as the answer, but the correct answer is -6.

4.2 Variable window

When does the right side move (expand)?

In every round of the loop. The element a[right] is added to the sum, so the window grows by one element on the right.

Why does the left side move (contract)?

When the sum becomes more than the limit, the window is not valid any more. To make it valid again, elements are removed from the left, one by one, and left moves forward, until the sum is within the limit. Removing from the left is correct because all the numbers are non-negative: a longer window starting at the same left would have an even bigger sum, so it can never work.

When is the best answer updated?

After the contraction, when the window is valid (the sum is within the limit and the window is not empty). If its length right-left+1 is more than the best length so far, the best length, start and end are replaced. If the length is the same or smaller, nothing changes.

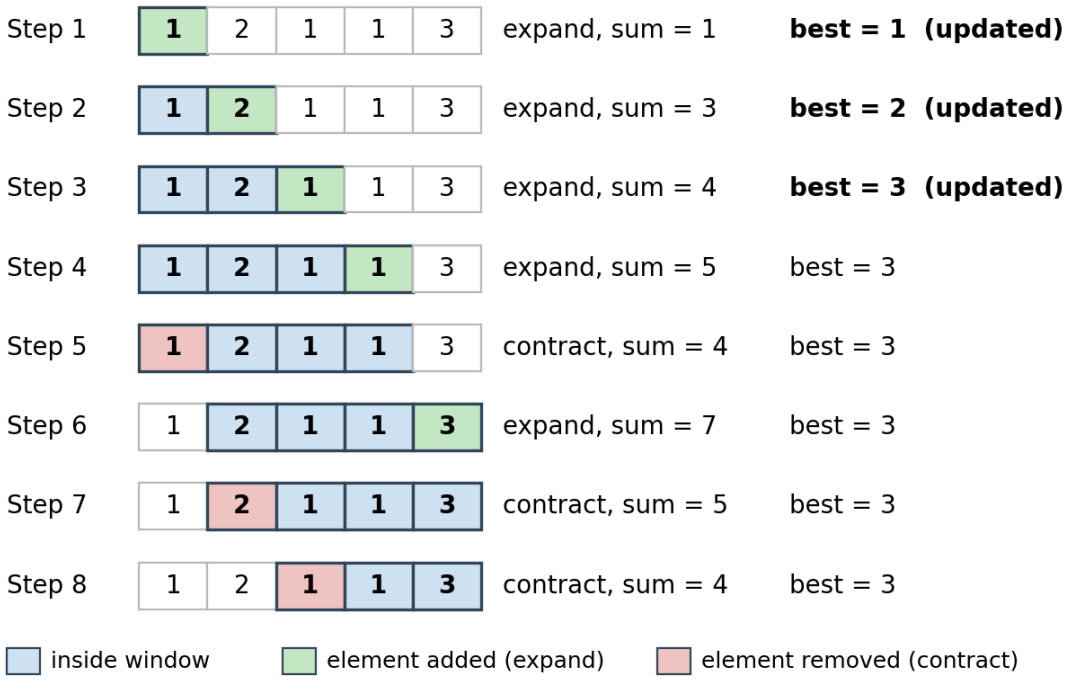


Figure 3: Variable window for a = [1,2,1,1,3] and limit = 4 (best length = 3)

The sliding window method is correct only for non-negative numbers. For example, take a = [5, -3, 1] and limit = 3. The whole array has sum 3, so the correct longest length is 3, but when I ran my sliding window function it returned length 2 (indices 1 to 2), because after removing 5 it never goes back. So this method should not be used when the array can have negative values.

5. Trace Tables

5.1 Fixed window

Step 1 is the first window, and each next step is one move of the window. The Window column gives the start and end index.

a = [2, 1, 5, 1, 3, 2], K = 3

Step	Window	Elements	Sum calculation	Running sum	Max so far
1	0-2	[2,1,5]	initial sum = 2 + 1 + 5	8	8
2	1-3	[1,5,1]	8 + (1) - (2)	7	8
3	2-4	[5,1,3]	7 + (3) - (1)	9	9
4	3-5	[1,3,2]	9 + (2) - (5)	6	9

Edge cases: all negative numbers, K = 1 and K = n

Case	Step	Window	Elements	Sum calculation	Sum	Max so far
a=[-4,-2,-7], K=2	1	0-1	[-4,-2]	initial sum = (-4) + (-2)	-6	-6
	2	1-2	[-2,-7]	-6 + (-7) - (-4)	-9	-6
a=[6], K=1	1	0-0	[6]	initial sum = 6	6	6

Case	Step	Window	Elements	Sum calculation	Sum	Max so far
a=[1,2,3], K=3	1	0-2	[1,2,3]	initial sum = 1 + 2 + 3	6	6

For [-4,-2,-7] the second window gives -9, which is less than -6, so the maximum stays -6. For K = 1 and K = n there is only one window, so it does not move.

5.2 Variable window

Every expand and every contract is one row. Sum, Length and left show the values after that step. The last two columns show the best answer found so far.

a = [1, 2, 1, 1, 3], limit = 4

Step	right	left	Action	Sum	Length	Best length	Best (start,end)
1	0	0	expand: add a[0]=1; best updated	1	1	1	(0,0)
2	1	0	expand: add a[1]=2; best updated	3	2	2	(0,1)
3	2	0	expand: add a[2]=1; best updated	4	3	3	(0,2)
4	3	0	expand: add a[3]=1	5	4	3	(0,2)
5	3	1	contract: remove a[0]=1	4	3	3	(0,2)
6	4	1	expand: add a[4]=3	7	4	3	(0,2)
7	4	2	contract: remove a[1]=2	5	3	3	(0,2)
8	4	3	contract: remove a[2]=1	4	2	3	(0,2)

The window contracts in steps 5, 7 and 8. In step 4 the sum becomes 5, which is more than 4, so a[0] is removed. In step 6 the sum is 7, so a[1] and a[2] are removed. The best answer is length 3 at indices 0 to 2.

Zero limit: a = [4, 2, 1], limit = 0

Step	right	left	Action	Sum	Length	Best length	Best (start,end)
1	0	0	expand: add a[0]=4	4	1	0	-
2	0	1	contract: remove a[0]=4	0	0	0	-
3	1	1	expand: add a[1]=2	2	1	0	-
4	1	2	contract: remove a[1]=2	0	0	0	-
5	2	2	expand: add a[2]=1	1	1	0	-
6	2	3	contract: remove a[2]=1	0	0	0	-

Every element is bigger than 0, so after each expand the window is emptied by the contract steps, and the best length stays 0.

6. Test Cases

Both methods were run on the minimum test cases of the assignment. In Part B more than one correct answer is possible, so a test is marked Pass when the length is as expected and the subarray at the returned indices has a sum within the limit. The indices are given in brackets.

No.	Problem	Input	Expected output	Reference output	Sliding window output	Result
1	Fixed window	a=[2,1,5,1,3,2], K=3	Max sum = 9	9	9	Pass
2	Fixed window	a=[-4,-2,-7], K=2	Max sum = -6	-6	-6	Pass
3	Fixed window	a=[6], K=1	Max sum = 6	6	6	Pass
4	Fixed window	a=[1,2,3], K=3	Max sum = 6	6	6	Pass
5	Variable window	a=[1,2,1,1,3], limit=4	Longest length = 3 (0-2 or 1-3)	3 (0-2)	3 (0-2)	Pass
6	Variable window	a=[5,1,1], limit=2	Longest length = 2 (1-2)	2 (1-2)	2 (1-2)	Pass
7	Variable window	a=[4,2,1], limit=0	Longest length = 0	0	0	Pass
8	Variable window	a=[0,0,1,0], limit=1	Longest length = 4 (0-3)	4 (0-3)	4 (0-3)	Pass

16 out of 16 runs passed (8 test cases, each run on 2 methods). The programs also print the expected value, both results and PASS or FAIL for each test.

An invalid K was tested separately with a = [1, 2, 3]. For K = 4 and K = 0, both Part A functions print "Error: Invalid window size K ..." and return the smallest integer (-2147483648 in C++ and Java, minus infinity in Python) instead of a sum.

7. Complexity Comparison

Method	Time	Auxiliary space	Where the cost comes from
Part A, simple	O(n*K)	O(1)	There are n-K+1 windows, and for each one the K numbers are added again from the start.
Part A, sliding window	O(n)	O(1)	K additions for the first window, then one addition and one subtraction for each of the n-K moves.
Part B, brute force	O(n^2)	O(1)	Each of the n starting indices can go up to the end of the array, so about n*n/2 steps in the worst case.
Part B, sliding window	O(n)	O(1)	right moves n times and left only moves forward, at most n times. Every element is added once and removed at most once.

Auxiliary space is O(1) for all four versions, because only a few variables (sum, left, right, best) are used and no new array is made. To see the difference, I counted the additions. For n = 1000 and K = 500, the simple method in Part A does 250500 additions, but the sliding window method does only 1000. For Part B with n = 1000 (all zeros and limit 1, which is the worst case for brute force), brute force does 500500 additions and the sliding window method does 1000 steps.

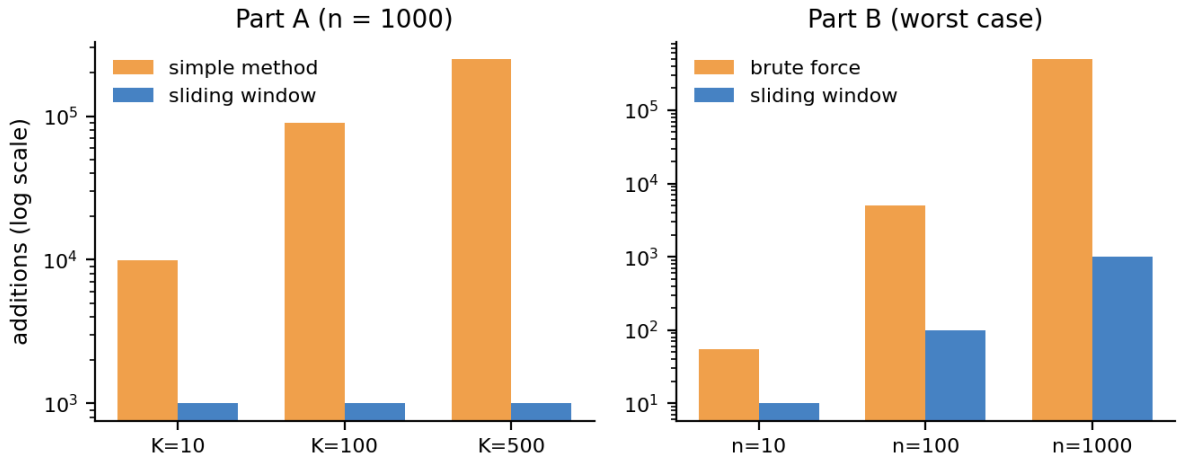


Figure 4: Number of operations, simple method compared with sliding window (log scale)

8. How to Compile and Run

```
C++:     g++ -std=c++17 -o assignment2 assignment2.cpp
         ./assignment2
Java:    javac Assignment2.java
         java Assignment2
Python:  python3 assignment2.py
```

The programs do not take any input. The test arrays are written inside the code, and the results are printed. This is the output of the C++ program (Java and Python print the same results):

```
============================================================
PART A: Fixed-Size Window (Maximum Sum of Subarray of Size K)
============================================================
Array: [2, 1, 5, 1, 3, 2], K = 3
  Expected Max Sum : 9
  Reference Result : 9
  Optimized Result : 9
  Status           : [PASS]

Array: [-4, -2, -7], K = 2
  Expected Max Sum : -6
  Reference Result : -6
  Optimized Result : -6
  Status           : [PASS]

Array: [6], K = 1
  Expected Max Sum : 6
  Reference Result : 6
  Optimized Result : 6
  Status           : [PASS]

Array: [1, 2, 3], K = 3
  Expected Max Sum : 6
  Reference Result : 6
  Optimized Result : 6
  Status           : [PASS]

============================================================
PART B: Variable-Size Window (Longest Subarray with Sum <= K)
============================================================
Array: [1, 2, 1, 1, 3], Limit = 4
  Expected Length : 3
  Reference Result: Length = 3 (indices 0 to 2)
  Optimized Result: Length = 3 (indices 0 to 2)
  Status          : [PASS]

Array: [5, 1, 1], Limit = 2
```

```
  Expected Length : 2
  Reference Result: Length = 2 (indices 1 to 2)
  Optimized Result: Length = 2 (indices 1 to 2)
  Status          : [PASS]

Array: [4, 2, 1], Limit = 0
  Expected Length : 0
  Reference Result: Length = 0
  Optimized Result: Length = 0
  Status          : [PASS]

Array: [0, 0, 1, 0], Limit = 1
  Expected Length : 4
  Reference Result: Length = 4 (indices 0 to 3)
  Optimized Result: Length = 4 (indices 0 to 3)
  Status          : [PASS]
```

9. Screenshots

```
ankitrajjha@Ankitrajs-MacBook-Air DSA 2 % cd "/Use
rs/ankitrajjha/DSA 2/Assignment2/" && g++ assignme
nt2.cpp -o assignment2 && ./assignment2
============================================================
Array: [2, 1, 5, 1, 3, 2], K = 3
  Expected Max Sum : 9
  Reference Result : 9
  Optimized Result : 9
  Status           : [PASS]

Array: [-4, -2, -7], K = 2
  Expected Max Sum : -6
  Reference Result : -6
  Optimized Result : -6
  Status           : [PASS]

Array: [6], K = 1
  Expected Max Sum : 6
  Reference Result : 6
  Optimized Result : 6
  Status           : [PASS]

Array: [1, 2, 3], K = 3
  Expected Max Sum : 6
  Reference Result : 6
  Optimized Result : 6
  Status           : [PASS]

============================================================
PART B: Variable-Size Window (Longest Subarray with Sum <= K)
============================================================
Array: [1, 2, 1, 1, 3], Limit = 4
  Expected Length : 3
  Reference Result: Length = 3 (indices 0 to 2)
  Optimized Result: Length = 3 (indices 0 to 2)
  Status          : [PASS]

Array: [5, 1, 1], Limit = 2
  Expected Length : 2
  Reference Result: Length = 2 (indices 1 to 2)
  Optimized Result: Length = 2 (indices 1 to 2)
  Status          : [PASS]

Array: [4, 2, 1], Limit = 0
  Expected Length : 0
  Reference Result: Length = 0
  Optimized Result: Length = 0
  Status          : [PASS]

Array: [0, 0, 1, 0], Limit = 1
  Expected Length : 4
  Reference Result: Length = 4 (indices 0 to 3)
  Optimized Result: Length = 4 (indices 0 to 3)
  Status          : [PASS]
```

Screenshot 1: C++: compile and run

```
ankitrajjha@Ankitrajs-MacBook-Air Assignment2 % cd "/Users/ankitrajjha/DSA 2/Assignment2/" && javac A
============================================================
PART A: Fixed-Size Window (Maximum Sum of Subarray of Size K)
============================================================
Array: [2, 1, 5, 1, 3, 2], K = 3
  Expected Max Sum : 9
  Reference Result : 9
  Optimized Result : 9
  Status           : [PASS]

Array: [-4, -2, -7], K = 2
  Expected Max Sum : -6
  Reference Result : -6
  Optimized Result : -6
  Status           : [PASS]

Array: [6], K = 1
  Expected Max Sum : 6
  Reference Result : 6
  Optimized Result : 6
  Status           : [PASS]

Array: [1, 2, 3], K = 3
  Expected Max Sum : 6
  Reference Result : 6
  Optimized Result : 6
  Status           : [PASS]

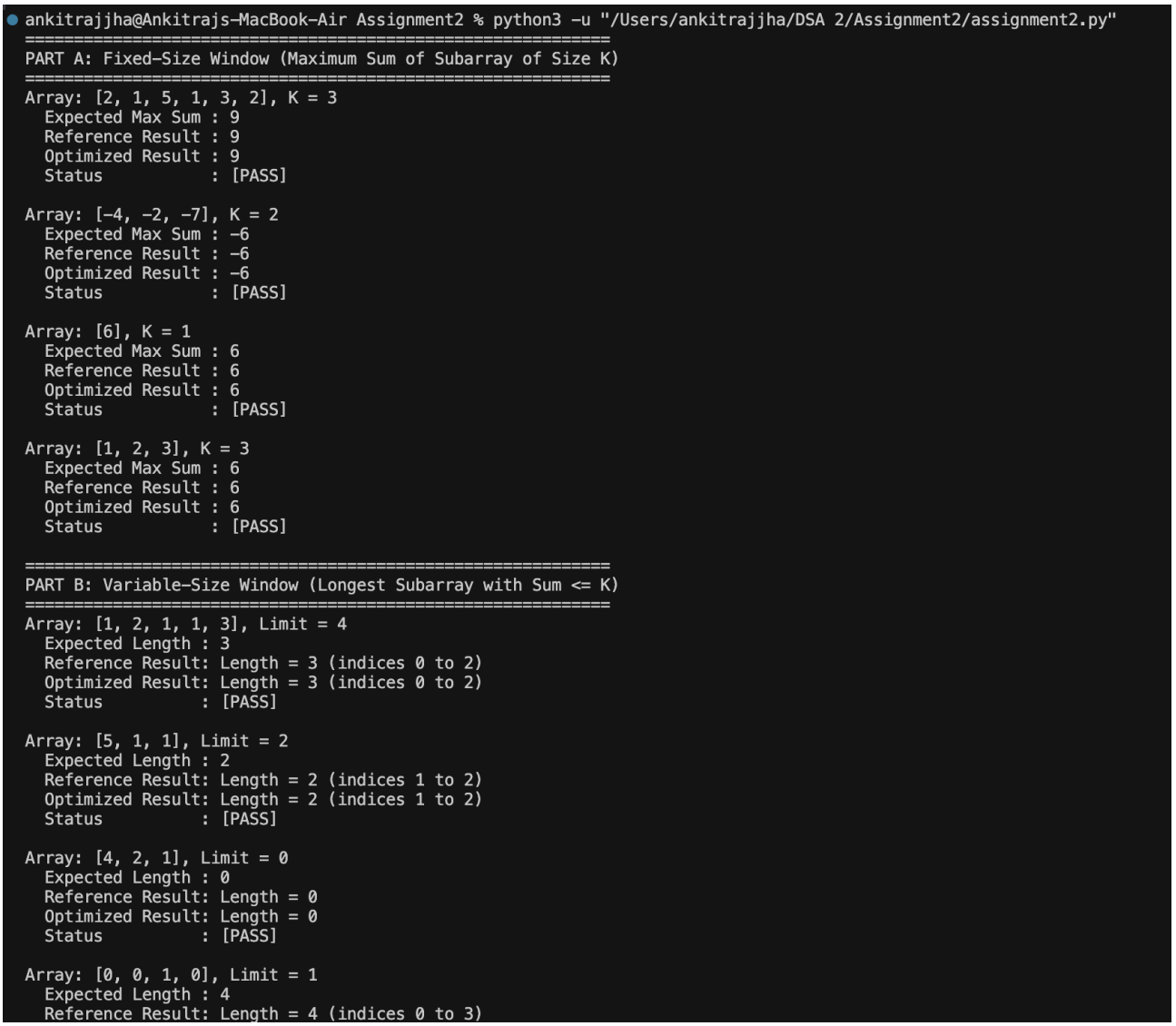
============================================================
PART B: Variable-Size Window (Longest Subarray with Sum <= K)
============================================================
Array: [1, 2, 1, 1, 3], Limit = 4
  Expected Length : 3
  Reference Result: Length = 3 (indices 0 to 2)
  Optimized Result: Length = 3 (indices 0 to 2)
  Status          : [PASS]

Array: [5, 1, 1], Limit = 2
  Expected Length : 2
  Reference Result: Length = 2 (indices 1 to 2)
  Optimized Result: Length = 2 (indices 1 to 2)
  Status          : [PASS]

Array: [4, 2, 1], Limit = 0
  Expected Length : 0
  Reference Result: Length = 0
  Optimized Result: Length = 0
  Status          : [PASS]

Array: [0, 0, 1, 0], Limit = 1
  Expected Length : 4
  Reference Result: Length = 4 (indices 0 to 3)
```

Screenshot 2: Java: compile and run



Screenshot 3: Python: run

10. Conclusion

I solved both problems with a simple method and with the sliding window method, and both gave the same correct answers on all the test cases (16 out of 16 runs passed). The sliding window method is faster because it reuses the earlier sum instead of adding the same elements again and again. Part A went from O(n*K) to O(n), and Part B from O(n^2) to O(n), with O(1) extra space.

Two things I learned: in Part A the maximum must start from the first window's sum so that arrays with all negative numbers work, and in Part B the sliding window method works only when the numbers are non-negative.